

LABMENTIX

Cybersecurity Training Program

CYBERSECURITY PROJECT REPORT

15-Day Hands-On Project Submission

Project Title	Caesar and Vigenere Cipher Tool
Student Name	Aharnish Dhaw Kumar
Batch / Cohort	15 August 2026
Domain	Cryptography
Trainer Name	Ganesh Kuche
Submission Date	22/09/2026
Difficulty Level	Advance

SECTION 1 — PROJECT ABSTRACT

1. Project Abstract

This project implements a custom Python-based cipher tool and graphical user interface to explore classical cryptography, focusing specifically on Caesar and Vigenère substitution ciphers. It addresses the fundamental security vulnerability of monoalphabetic and polyalphabetic ciphers, which preserve the underlying letter-frequency distribution of plaintext languages. By building automated cracking utilities utilizing chi-squared statistical scoring, Kasiski examination, and Index of Coincidence metrics, the project demonstrates how structural patterns expose hidden plaintexts without requiring prior knowledge of the key. The findings highlight the exact statistical weaknesses that modern encryption standards, such as AES, are engineered to eliminate.

SECTION 2 — INTRODUCTION

2. Introduction

2.1 Background

Cryptography is a foundational pillar of data security, utilizing mathematical algorithms to protect sensitive information from unauthorized access. Historically, classical substitution ciphers like Caesar and Vigenère served as primary methods for securing correspondence. Analyzing how these classical algorithms operate and why they fail provides essential insight into core security concepts like frequency distribution, structural entropy, and cryptographic vulnerability analysis.

2.2 Problem Statement

Classical substitution ciphers fail to obscure the inherent statistical structure of a language. Because letters in natural text appear with predictable frequencies (such as 'e' and 't' being common and 'q' and 'z' being rare), an attacker can easily analyze ciphertexts using statistical methods like chi-squared scoring, Kasiski examination, and Index of Coincidence to recover hidden messages and keys without authorization.

2.3 Objectives

- To design and develop a command-line and graphical tool for Caesar and Vigenère encryption, decryption, and analysis.
- To implement chi-squared frequency analysis to automatically crack Caesar shift ciphers.
- To integrate Kasiski examination and Index of Coincidence calculations for Vigenère key-length detection.
- To test the application across various ciphertexts and document performance observations.

SECTION 3 — TOOLS & ENVIRONMENT

3. Tools and Environment Setup

Tool / Software	Version / Platform	Purpose
Python 3	Version 3.10+	Core logic and cryptographic implementation language
Tkinter	Python Standard Library	Graphical user interface presentation layer
Visual Studio Code / Terminal	Native OS	Development, testing, and script execution environment
Git / Markdown	Latest	Code versioning and documentation management

3.1 Lab / Environment Description

The project environment was deployed locally on a machine running Python 3. The architecture is split into modular components: `cipher_tool.py` handles core algorithms, frequency analysis statistics, and command-line interactions; `cipher_gui.py` provides an interactive Tkinter graphical interface built on top of the core logic; and `README.md` documents usage instructions and theoretical background.

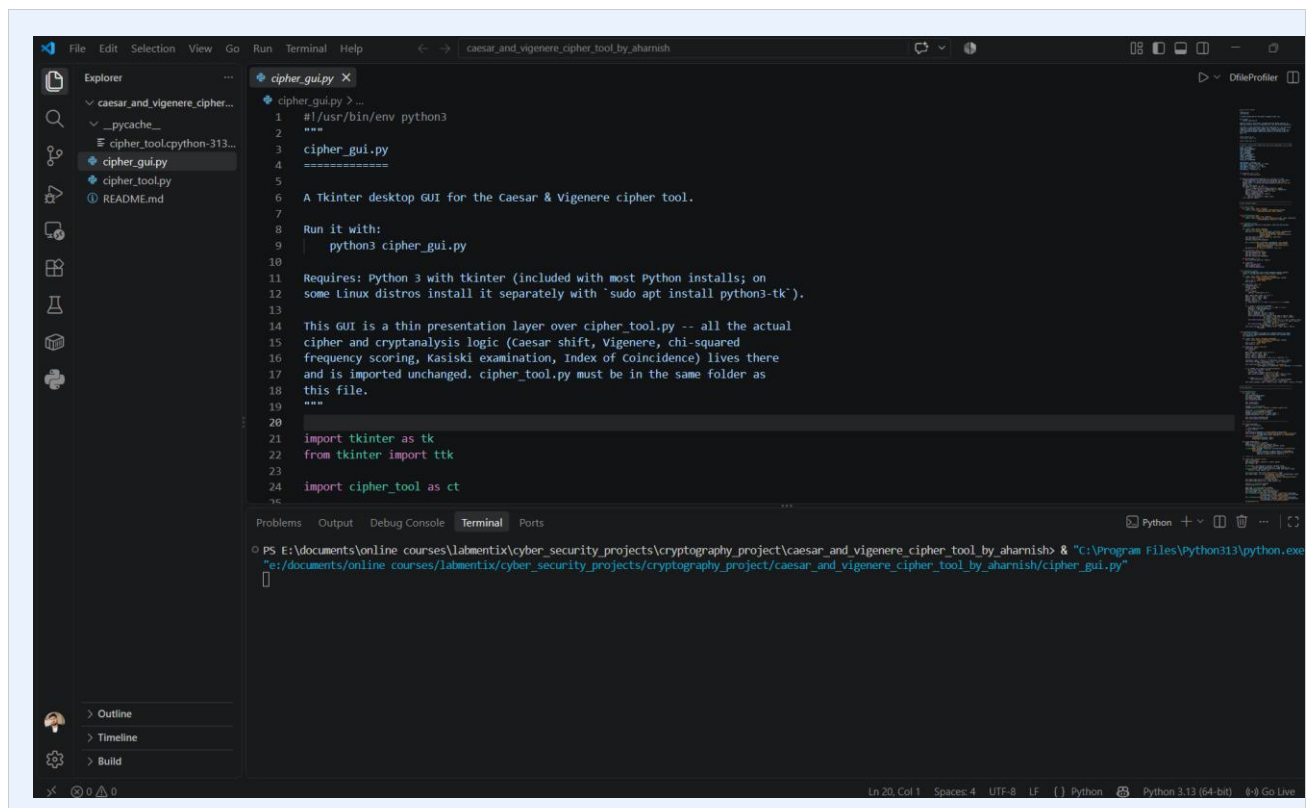
SECTION 4 — METHODOLOGY / STEP-BY-STEP PROCESS

4. Methodology

Step 1 — Verify Core Code and Run Interactive Demo

The first step involved validating the backend cipher logic and automated routines by running the built-in demonstration script via the command line. This confirms that encryption, decryption, and frequency distribution calculations function as intended.

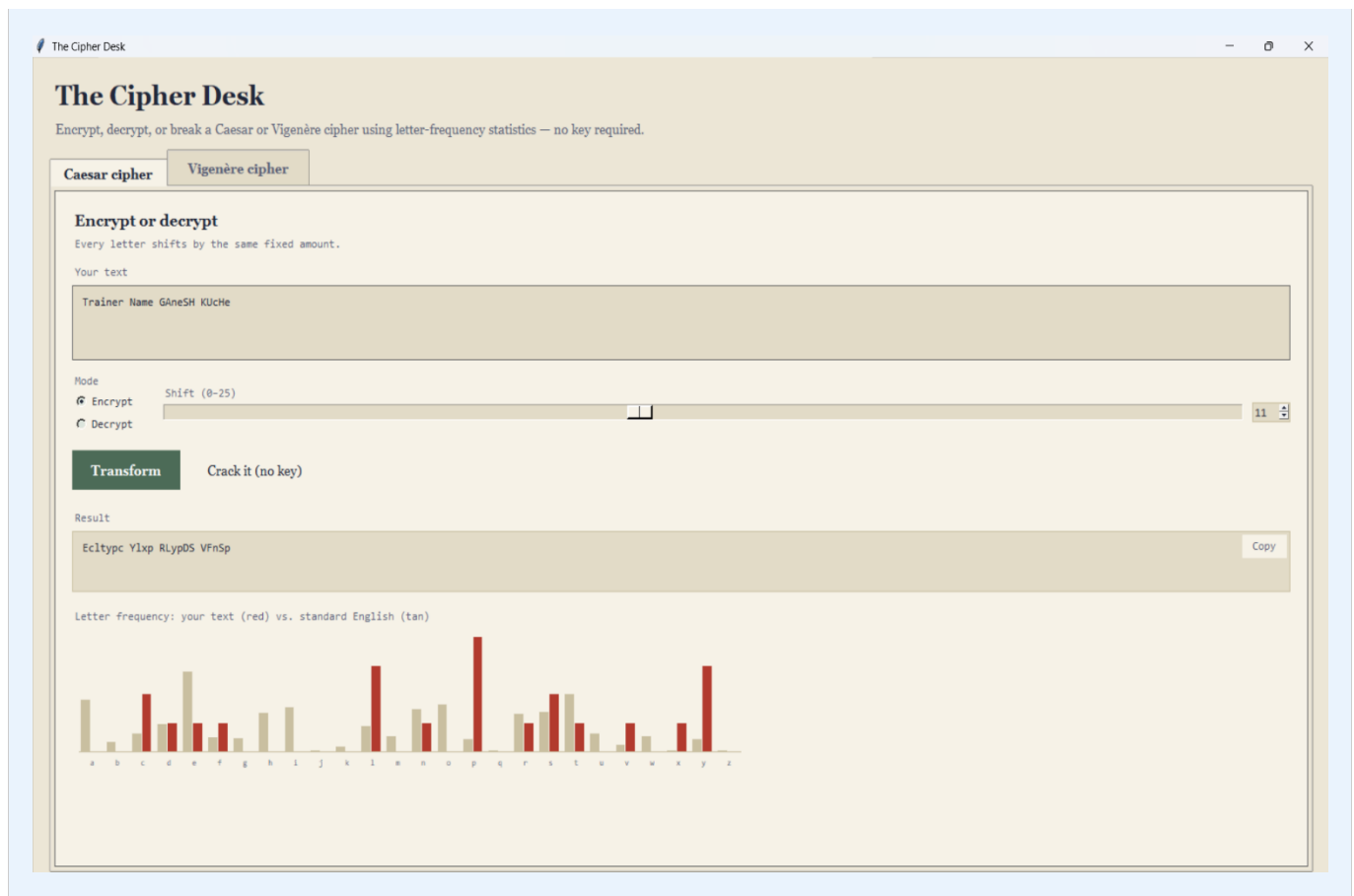
```
python3 cipher_tool.py demo
```



Step 2 — Execute Automated Caesar Cipher Cracking

Next, the chi-squared frequency analysis function was tested against a Caesar-encrypted ciphertext to verify that the tool can automatically determine the correct shift value without knowing the key.

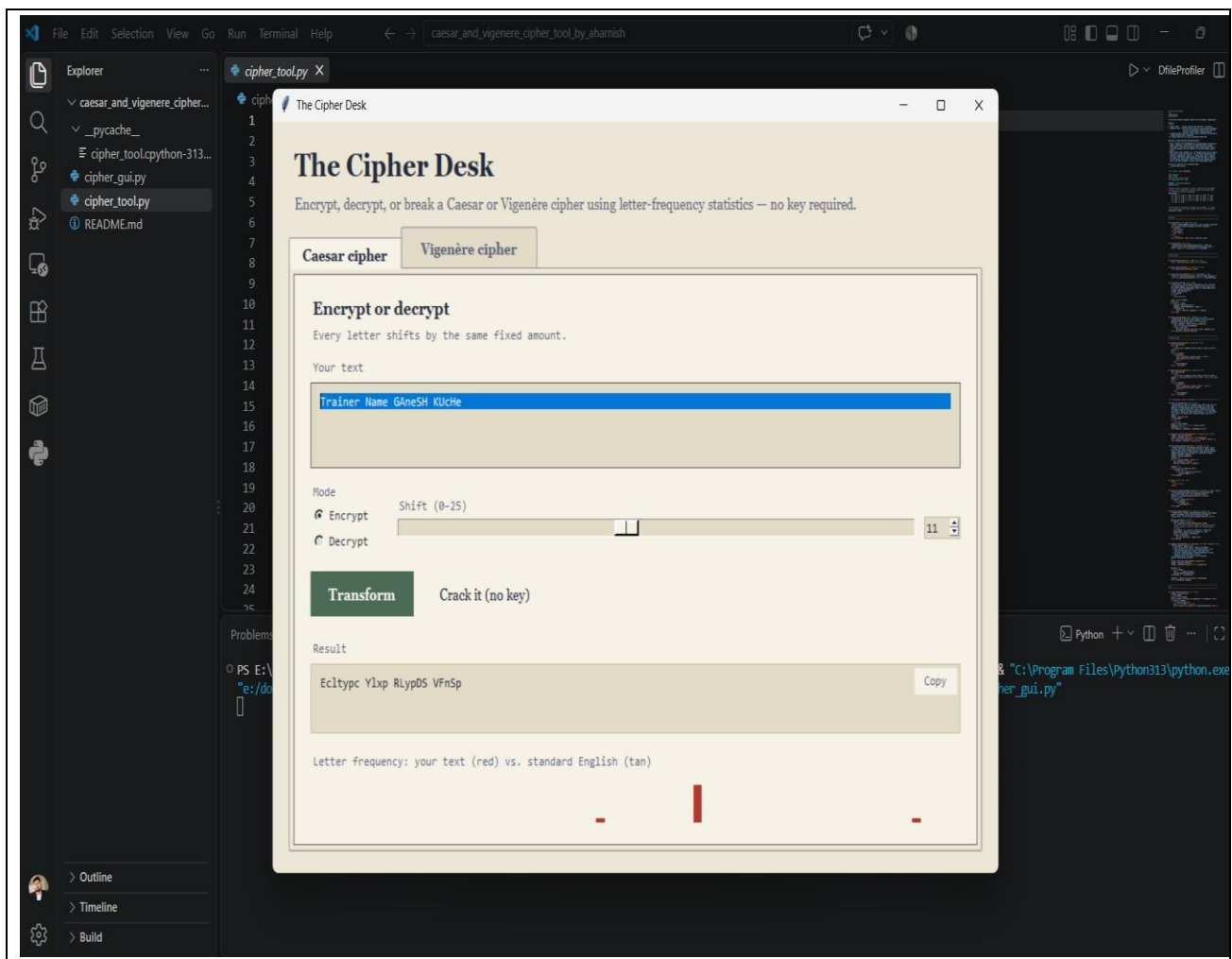
```
python3 cipher_tool.py caesar-crack "Eclytpc Ylxp RLypDS VFnSp"
```



Step 3 — Launch the Desktop Graphical Interface

To provide a visual layer, the Tkinter desktop GUI was launched. The interface imports cipher_tool.py directly to process inputs in real-time across both Caesar and Vigenère cipher panels.

```
python3 cipher_gui.py
```



Step 4 — Perform Vigenère Auto-Cracking and Statistical Analysis

A longer multi-word ciphertext sample was entered into the Vigenère tab to trigger Kasiski examination and Index of Coincidence analysis. The tool evaluates candidate key lengths, generates an IoC chart with an English threshold reference, extracts the keyword, and decrypts the message.

The Cipher Desk

Encrypt, decrypt, or break a Caesar or Vigenère cipher using letter-frequency statistics — no key required.

Caesar cipherVigenère cipher

Encrypt or decrypt

The keyword repeats, letter by letter, choosing a different shift each time.

Your text

Trainer Name GÁnESH KUCHe LABMentix
attack at dawn

ModeKeyword

Encrypt

Decrypt

lemon

TransformCrack it (no key)

How the keyword lines up

LEMONLEMONLEMONLEMONLEMONLEMONLEMONLEMONLEMONLE
TRAINERNAMEGANESHKUCHELABMENTIXATTACKATDAMN

Result

Evmwqv Zozp K%brDL WIpSi X0oXizhvi
efhno mh qlaz

Copy

SECTION 5 — RESULTS & OBSERVATIONS

5. Results and Observations

5.1 Summary of Findings

#	Finding / Observation	Details	Severity / Impact
1	Chi-Squared Efficiency	Successfully identified correct Caesar shifts by locating the lowest statistical variance against standard English letter frequencies.	High
2	Text Length Dependency	Short ciphertexts produced unstable Index of Coincidence metrics, demonstrating that statistical cracking requires sufficient data volume.	Medium
3	Kasiski Repetition Accuracy	Repeated substrings in Vigenère ciphertexts reliably pointed to multiples of the keyword length.	High

5.2 Analysis

The results confirm that classical substitution ciphers are inherently weak against frequency analysis. Because the substitution mapping is consistent or cyclically repeating, statistical signatures survive encryption. This directly contrasts with modern symmetric ciphers like AES, which use confusion and diffusion layers to completely flatten letter-frequency distributions.

SECTION 6 — CHALLENGES FACED

6. Challenges Faced

Challenge Faced	How I Solved It
Handling Short Text Blocks	Automated cryptanalysis failed on very short strings because statistical averages require a larger sample size. Resolved by incorporating conditional checks that prompt users for longer ciphertext samples during automated Vigenère cracking.
GUI Module Integration	Ensuring clean imports between the Tkinter GUI file and the backend CLI script. Resolved by structuring both files within the same working directory and keeping the GUI strictly as a presentation layer.

SECTION 7 — LEARNING OUTCOMES

7. Learning Outcomes

- Learned how to calculate and apply chi-squared statistics to match text letter frequencies against standard English distributions.
- Gained deep practical understanding of Kasiski examination and Index of Coincidence formulas to isolate polyalphabetic key lengths.
- Understood the architectural design principles separating core security logic from graphical presentation layers in Python.
- Realized the foundational security differences between classical substitution ciphers and modern diffusion-based encryption standards.

SECTION 8 — CONCLUSION & FUTURE SCOPE

8. Conclusion

This project successfully achieved its objectives by building a fully functional cryptographic analysis and transformation tool. The implementation proved that classical ciphers can be easily dismantled using automated statistical frequency checks. The findings underscore why modern cryptography relies heavily on complex mathematical rounds, nonces, and diffusion blocks to prevent structural analysis.

8.1 Future Scope

Future enhancements could include adding graphical frequency distribution curves directly into the GUI, supporting additional classical ciphers like the Playfair or Hill cipher, and integrating educational modules illustrating modern AES round transformations.

SECTION 9 — REFERENCES

9. References

- [1] Labmentix Cybersecurity Training Program — Project Submission Guidelines
- [2] Python Standard Library Documentation: collections and tkinter — <https://docs.python.org/3/>
- [3] Classical Cryptography and Cryptanalysis Textbooks (Kasiski Examination, Index of Coincidence, and Chi-Squared Statistics)

DECLARATION

Student Declaration

I hereby declare that this project report is my own original work, completed as part of the Labmentix Cybersecurity Training Program. The work presented in this report is genuine, and all external references have been duly acknowledged.

Aharnish Dhaw Kumar

Student Signature

22/09/2026

Date of Submission